PART 4A — JAMES CABREROS — PROCESS IMPROVEMENT AND PLACEMENT ARCHITECTURE

Governed process design: promise policy, operating mechanisms, enrollment, and infrastructure flow

James Cabreros — 11:00 AM slot

Senior Manager, Supply Chain · placement architecture · process improvement · configuration governance

How to use this file. J1–J3 are Walmart Marketplace design cases. J4 is one end-to-end Google Fiber infrastructure supply-chain design followed by five zoomed STARs. Every design result is a proposed mechanism or a labeled model—not a production claim. Each is paired with a lived failure or run-experience analog so I can move to evidence if the interviewer asks, “Have you actually seen this break?”

Contents

J1 Walmart promise buffer — one governed dial

J2 Walmart promise-yield WBR + Andon

J3 Walmart managed-shipping enrollment funnel

J4 Google Fiber — one end-to-end infrastructure supply-chain map

J4A E3 + E11 vendor EDI/VMI → direct-procurement PO lifecycle

J4B E1 + E2 contractor truck stock → serial traceability

J4C E5 3PL reconciliation — “two books, one truth”

J4D E10 returns/RMA — escape routing by disposition

J4E E12 consignment/vending ledger — capacity signaling

Integrity key. record = lived source; **record-from-docs** = source artifact; **modeled** or **illustrative** = design math only. I designed and governed; client engineering/SAP teams built or would build. I do not claim I configured SAP, ran VMI, or delivered these design-case outcomes.

DESIGN CASE — NOT A DELIVERED RESULT

J1 · Walmart pre-purchase delivery experience — the governed promise-buffer dial (S19)

Placement-architecture hook: policy before configuration; one auditable business dial, shadow validation, peak regime change.
Paired lived record: Apple F4 / Elementum lead-time distributions and the real in-transit promise-rule miss.

60-SECOND PITCH

S — In a Walmart Marketplace design engagement, the same flat 1.10-day buffer sat on every seller/carrier cohort. On the illustrative twelve-cohort book it looked healthy at 95.8% accuracy, but it over-insured tight lanes and under-insured volatile ones. **T —** I had to turn a hidden model parameter into a governed business policy without claiming I built the promise engine. **A —** I decomposed the promise, compared four policies, recommended a risk-proportional buffer $b = k\sigma$, made k the one named dial, and designed four shadow weeks, a 95% promotion floor, one-flag rollback, and a pre-peak re-solve. **R —** In the illustrative model, fast-promise share moved 7.6%→39.6%, +32 points, at the same 95% floor; average promised day moved 3.09→2.75. That is modeled, not banked. The delivered artifact was the policy table, gate, and governance mechanism.

S — Situation. I was leading the promise-policy workstream for a marketplace design in which the customer saw one date but responsibility was split across seller handling, carrier transit, and a residual buffer. A flat buffer had no active owner, and a green portfolio average hid opposite cohort-level failures.

T — Task. I owned the decomposition, alternatives, test gates, and operating policy. I had to preserve a 95% modeled accuracy floor while recovering speed, and make the trade auditable by a business owner. Client data science and engineering owned the model and engine.

A — Action. I ran five gates: (1) **discover**—audit early-delivery waste and variation by cohort; (2) **design**—compare uniform, risk-proportional, minimum-days, and profit-optimal policies on one book; (3) **test**—prove that $k=1.645$ holds the 95% illustrative floor by cohort and stress Q4 variability; (4) **cut over**—compute the challenger in shadow for four consecutive weeks and keep the incumbent warm; (5) **govern**—publish k, p90 error, exceedance, and waste in the weekly header, with a named business owner and a peak reset date.

R — Result. The illustrative model produced 7.6%→39.6% fast-promise share and 3.09→2.75 average promised day at the held 95% floor. The economic line was about $255K/week, or about $13.3M annualized, **modeled** on an illustrative million-order weekly book; there is no production dollar. The real process result I claim is one signable policy, one reversible promotion gate, and one explicit owner.

TRADE-OFF I MADE. I chose one governed, risk-proportional dial over per-lane learned buffers. I knowingly left some modeled value on the table to gain explainability, auditability, and a one-flag rollback; volatile cohorts received slower promises on purpose so tight cohorts could become faster without spending the accuracy floor.

LIVED FAILURE / LEARNING. In my real inventory-promise work, my first in-transit rule was too conservative and starved dotcom of sellable units for about a week. I replaced the blanket rule with a per-lane promise window backed by observed lane data. That failure is why this design shadows against real orders and governs by cohort rather than trusting a portfolio average.

LP tags. Dive Deep—I went below a green 95.8% portfolio print to cohort variance. · **Invent and Simplify**—I reduced many micro-decisions to one governed dial. · **Customer Obsession**—I optimized the promise kept, not merely the date displayed. · **Have Backbone; Disagree and Commit**—I rejected a more opaque per-lane model for the first release.

SAFE-SAY. “The +32 points and all dollars are from the engagement’s illustrative twelve-cohort model; I claim the decision instrument, not a production outcome.”
“My source is master story S19; the lived analog is the signed inventory-promise work and its documented in-transit-rule correction.”

20-SECOND VERSION

I took a flat promise buffer that was simultaneously too slow and too risky and designed one risk-proportional dial, shadowed for four weeks, promoted only at a 95% floor, rolled back in one flag, and reset before peak. The illustrative model bought +32 points of fast-promise share; what I claim is the governed mechanism, not production P&L.

FOUR PROBES

1 · Why not optimize each lane independently?

That is a higher modeled frontier but a weaker first operating policy. I started with one explainable dial; I would graduate to lane-level tuning only after shadow evidence showed material incremental value and Finance accepted the breach economics.

2 · What stops a peak-season accuracy collapse?

The dial is regime-specific. The Q4 drill showed a frozen summer setting breaking the floor, so the owner must re-solve and publish the peak value before October; breach of the shadow floor blocks promotion.

3 · What exactly did you own?

The promise identity, waste audit, four-policy decision table, stress proof, promotion/rollback gates, and governance handoff. Client teams owned the production model and engine.

4 · Where is the process-improvement mechanism?

Configuration can no longer drift silently. Every change has an owner, a visible dial, a cohort-level tail read, a shadow result, a promotion decision, and a rollback path.

CONFIGURATION-GOVERNANCE HANDOFF CARD

CONTROL	BEFORE DEPLOYMENT	AFTER DEPLOYMENT
Policy	Owner signs floor, cohort grain, and peak regime.	Dial and model version appear in every review.
Validation	Four shadow weeks; cohort tails clear the floor.	Waste, p90 error, exceedance, and contacts trend together.
Change	One flag; incumbent remains warm; rollback trigger written.	Breach rolls back, starts cause review, and blocks expansion.
Correction	Peak drill before launch.	Pre-peak re-solve becomes a dated annual control.

DESIGN CASE — OPERATING MECHANISM, NOT A PRODUCTION CLAIM

J2 · Walmart weekly business review + Andon for promise yield (S20)

Configuration-governance hook: outputs over owned inputs; one definition, denominator, owner, floor, action, and verification loop. **Paired lived record:** the Flexport weekly review I ran and my lived scheduler/status-control failures.

60-SECOND PITCH

S — Walmart's pre-purchase promise was split across seller, carrier, model, and engine teams; 30-day windows hid a degrading seller-lane cell and a red week became a blame meeting. **T —** I had to design the operating mechanism, not another dashboard. **A —** I established one glossary and event source, seven customer outputs over ten owned inputs, Promise Yield as fast share × accuracy, a factor decomposition that names seller, carrier, or buffer, Tuesday pre-wires, mix-adjusted twins, and a seven-day cell-level Andon that acts within 24 hours while keeping the 30-day clock for seller consequences. **R —** The design moved attribution from negotiation to minutes and detection-to-action from up to 30 days to under 24 hours. The sample Promise Yield baseline was 41.5%; about $0.58M per caught episode is modeled, never annualized.

S — Situation. I found outputs owned by a committee, three date clocks, denominator drift, and aggregate carrier performance that could look green nationally while one region failed customers.

T — Task. I owned the metric architecture, thresholds, causal roll-up, meeting rhythm, and control loop. Client engineering owned pipelines; business owners had to own levers after handoff.

A — Action. I gated the mechanism: **discover** one definition, owner, grain, date basis, and denominator per metric; **design** seven outputs over ten turnable inputs, hard reds on policy floors, and one joint headline; **test** the factor identity on six sample weeks and inject a Southeast carrier degradation; **launch** Monday freeze, Tuesday decision, every red pre-wired with cause/owner/date; **control** with raw and mix-adjusted twins plus a seven-day EWMA/CUSUM Andon at seller-lane grain, sustained 2× baseline and at least 30 orders, reversible badge/buffer action within 24 hours.

R — Result. On the sample drill, factor attribution took minutes rather than days. The proposed control loop acted in under 24 hours instead of waiting on a 30-day window. Promise Yield was 43.6% fast share × 95.2% accuracy ≈ 41.5% **sample baseline**. The ~$0.58M episode line is **modeled** from three weeks earlier action; I do not claim a live output move.

TRADE-OFF I MADE. I kept the stable 30-day clock for seller consequences but used a fast seven-day clock for reversible customer protection. I rejected a larger board and instant punitive action: speed belongs at cell/badge level; fairness belongs at scorecard level.

LIVED FAILURE / LEARNING. In a real peak incident, a scheduler failed silently and the business detected it from aged promises. I added a heartbeat and "detect within one interval" rule. In the production Flexport WBR, I also learned that two disagreeing numbers must be reconciled before comparison. Those failures are why this design has freshness, denominator, and heartbeat controls—not just colored tiles.

LP tags. Insist on the Highest Standards—hard floors can stop expansion. · **Earn Trust**—definitions and denominators are visible. · **Dive Deep**—factor math attributes the miss. · **Ownership**—every red has one owner, date, and verify step.

SAFE-SAY. "41.5% is the engagement sample baseline; the sub-24-hour Andon and ~$0.58M per episode are designed/modeled, not a live Walmart result."
"The source is master S20; my lived operating analog is the production WBR and the scheduler-heartbeat corrective action."

20-SECOND VERSION

I designed a promise WBR where every output decomposes into a turnable input with one owner, every red arrives pre-wired, and a cell-level Andon protects customers in under 24 hours while the fair 30-day seller clock stays intact. The engagement proved attribution on sample weeks; I do not claim live output movement.

FOUR PROBES

1 · Why Promise Yield?

Accuracy alone rewards slow promises; fast share alone rewards broken ones. Their product makes the trade visible, while the next line decomposes it so the composite cannot hide a cause.

2 · How do you avoid noisy Andons?

A sustained control signal, a 30-order floor, replay against stable cells, and a designed false-trip bar under 5%. The action is reversible and auto-restores on recovery.

3 · Who can stop a rollout?

A hard policy-floor red stops cohort expansion; the Andon can pull a cell's fast badge; the output owner can hold a launch that changes a metric definition.

4 · What if an input moves but the output does not?

The verify step declares the causal map wrong and reopens it. A review that cannot falsify its own model becomes numerology.

DESIGN CASE — WAVE ONE MECHANICS; ADOPTION CURVE UNCLAIMED

J3 · Walmart stalled managed-shipping program — enrollment-funnel process redesign (S21)

Self-service/configuration hook: diagnose the funnel, make the contract explicit, verify configuration in shadow, graduate cohorts on counted gates. **Paired lived record:** Elementum lane onboarding and the real "instrumented too late" miss.

60-SECOND PITCH

S — Walmart's managed shipping settings had stalled at roughly 38–42% of seller-fulfilled GMV. Sellers declared one-day handling but hit it about 85% of the time, while enrollment asked them to surrender control for a benefit they could not price. **T —** I had to redesign the process from "market the feature harder" to a rational, measurable exchange. **A —** I mapped enrolled/refused/churned sellers through the funnel, reframed the offer as transit automation + observed handling + scorecard protection, designed a personalized benefit line, shadow-ran each seller on its own history, and boarded waves only when handling coverage, inventory-feed quality, and protection rules cleared. I paired activations with 30-day retention. **R —** The design delivered the signed contract and wave-one mechanics; ≈$3.7M/year per ten points of enrolled GMV was **modeled**. I do not claim the later adoption curve.

S — Situation. I saw a process treated as a communications problem even though the refusal was economically rational: loss of carrier control, fear of cost, and distrust that the scorecard would honor carrier-delay protection.

T — Task. I owned the refusal diagnosis, contract structure, shadow gate, wave graduation, and measurement pair. Client product/engineering owned Seller Center and observed-handling computation.

A — Action. I interviewed enrolled, refused, and churned sellers and named the biggest funnel drop; wrote a bundle with explicit rights and obligations; replaced self-declared handling with observed scan history; made benefit visible in the seller's own modeled GMV; shadowed recent orders before flipping configuration; published wave criteria—settings verified, handling coverage, node inventory feed, protection armed—and stopped the next wave on any miss; measured activations and 30-day retention together, with a kill condition if scorecard protection failed.

R — Result. The claimed output is a signed design, one gated wave, and an honest measurement pair. The baseline 38–42% and 85% are **record-from-docs**. The ≈$3.7M/year per +10 points is **modeled** from handling-miss reduction; no post-wave adoption result is claimed.

TRADE-OFF I MADE. I favored carrots and tiered protection over an aggressive all-or-nothing enforcement cliff. That made the contract less pure and slower to standardize, but protected assortment and seller trust while the client's own fulfillment alternative was not yet a credible substitute.

LIVED FAILURE / LEARNING. In my real thousand-lane onboarding, I measured completeness from day one but instrumented tender acceptance only from cohort three; that delayed discovery of a carrier-data problem by two cohorts. Here, acceptance/retention and refusal codes exist before wave one. I learned that a rollout gate added after launch is only a report.

LP tags. Customer Obsession—observed handling protects the buyer's promise. · **Earn Trust**—seller protection is explicit and testable. · **Think Big**—the offer can scale through self-selection, not persuasion. · **Deliver Results**—waves graduate on evidence, not calendar.

SAFE-SAY. "The 38–42% and 85% are engagement-document baselines; ≈$3.7M is modeled per ten enrolled-GMV points. I claim the contract and wave mechanics, not the subsequent adoption curve."
"The source is master S21; the lived gate lesson is Apple F1's cohort onboarding record."

20-SECOND VERSION

I treated stalled adoption as an incentive and process problem: explicit insurance-like protection, observed rather than declared handling, seller-specific value, shadow verification, and waves gated on configuration and feed quality. Activations are paired with 30-day retention. Wave-one mechanics are the result; the later curve is not mine to claim.

FOUR PROBES

1 · Why did sellers refuse?

They traded away carrier and configuration control for protection that was real but weakly specified and unpriced. More banners would not change that contract.

2 · What is the pre-deployment validation?

Replay recent seller orders under observed handling and managed transit; compare projected accuracy and badge eligibility; arm scorecard protection; then flip only inside tolerance.

3 · What metric cannot be gamed?

Activations plus 30-day retention and protected-GMV share. Activations alone turn churn into apparent growth.

4 · When do you stop enrollment?

If the scorecard penalizes a carrier delay the contract promised to excuse, enrollment stops until the protection path reconciles. Trust is an invariant, not a follow-up ticket.

WAVE-GRADUATION CARD

GATE	PASS EVIDENCE	FAILURE ROUTE
Configuration	Observed handling and managed-transit replay within tolerance.	Keep seller on current settings; fix cause.
Connectivity	Node inventory feed complete and fresh.	Block fast badge and next-wave boarding.
Contract	Scorecard protection armed and tested.	Stop enrollment; reconcile protection path.
Adoption quality	Activation, 30-day retention, and protected GMV read together.	Refusal/churn reason re-enters design backlog.

DESIGN CASE — ONE END-TO-END PROCESS MAP

J4 · Google Fiber infrastructure supply chain — placement, connectivity, capacity, and escape routing

North-star invariant: every serialized unit is in exactly one state, at one location, under one owner; every transition has one durable document; every system-to-system hop reconciles. **Voice:** I would own the process, contracts, gates, and governance; SAP/CRM/3PL teams configure and build.

STAGE	FLOW / DOCUMENT / CONTROL	CONFIGURATION GOVERNANCE	JAMES-BULLET MAPPING
1 · E3 signal	Vendor 830/852 forecast/activity → 855 dated confirmation → 856 ASN + serials; VMI min/max from measured lead-time tails.	≥95% valid-ASN cohort gate; undated ack = unconfirmed; weak vendor stays on portal.	Inbound connectivity; capacity signaling.
2 · E11 buy	Build plan/MRP → PR → approved direct PO/850 → confirmed date → ASN → receipt/101 → invoice/810.	Direct stock/project stock in SAP; indirect spend outside stock; PO changes require reconfirmation.	Capacity signals for new node types.
3 · E1/E2 deploy	3PL → contractor/van stock → install by serial → CRM asset; crossdock bypasses storage for node kits.	Van as governed location; scan every transfer; install from wrong van holds; offline events replay idempotently.	Outbound connectivity; crossdock enablement.
4 · E4 promise	CRM order → SAP fulfillment → 3PL/field status → CRM; correlation id follows every hop.	One writer/status; unknown never maps to shipped; daily sent=created=delivered=status-returned.	Placement/order architecture.
5 · E5 reconcile	3PL 846 + receipts/ship confirms vs SAP by SKU×location×state×serial; “two books, one truth.”	Reason code timing/mapping/status/physical; lower-of-two while unresolved; only physical becomes adjustment.	Post-deploy validation; auditability.
6 · E10 escape	Return/RMA → untested → premise/central test → refurb A/B | repair | RTV | scrap.	No test/no grade/no promise; warranty and vendor-RMA checks route the exception.	Escape routing.
7 · E12 settle	Consignment receipt creates custody; install/dispense creates liability; refill/count; monthly settlement.	Append-only event ledger; count changes book only by document; capacity/route signal from par and stockout risk.	Capacity signaling; node configuration.

20-SECOND ARCHITECTURE ANSWER

I would run one serial-grain spine from vendor signal to PO, 3PL, van, CRM asset, return disposition, and settlement. Each hop has one owner and durable document; each node type has a configuration template and validation gate; E5 reconciles the physical and financial books; E10 is the governed escape route; E12 turns consumption into the capacity and liability signal.

DESIGN CASE — E3 + E11 · INBOUND CONNECTIVITY + CAPACITY SIGNALING

J4A · Vendor EDI/VMI → direct-procurement PO lifecycle for long-lead equipment

Lived analog: Elementum EDI onboarding—one thousand lanes requested, the 597 carrying 80% of spend first; ≥95% data-quality cohort gate. **Honesty:** I have run vendor/transport EDI onboarding, not a VMI program; VMI/consignment is the designed pattern.

60-SECOND PITCH

S — In the Google Fiber design, node equipment has 12–40-week lead times, confirmations in email, and build crews exposed to supplier variance. **T —** I would create one demand-to-receipt process whose dates harden as evidence improves and whose vendor signal can drive capacity. **A —** I would connect build-plan need to PR, approved PO/850, dated 855 confirmation, serialized 856 ASN, receipt, and 810 invoice; publish requested-versus-confirmed gap; rank expediting by milestone at risk; and, for strategic suppliers, share 830/852 demand/activity with min/max based on measured lead-time tails. A vendor cannot activate until valid ASNs clear 95%; undated acknowledgments remain unconfirmed. **R —** The proposed bar is ≥95% on-time to the vendor's confirmed date and zero PO lines unconfirmed after 48 hours. Any dollar is illustrative; the mechanism produces a capacity signal leaders can trust.

S — Situation. I would start with scattered demand, allocation risk, email confirmations, and requested dates incorrectly treated as supply. That creates false capacity at new nodes and heroic expediting.

T — Task. I would own the cross-functional process, document contract, activation gate, vendor scorecard, and exception ownership. Procurement owns the commercial terms; Planning owns demand; SAP and integration teams configure.

A — Action. I would define direct stock/project stock in SAP and keep services/expensed spend outside inventory; require PR→release→PO with no spreadsheet bypass; turn 855 date/quantity into the MRP-relevant supply; expose every line without a dated confirmation after 48 hours; validate serialized ASNs and acknowledgments before activation; sequence suppliers by criticality and spend; measure requested-date gap separately from confirmed-date reliability; for qualified VMI suppliers, send forecast/activity and set min/max from measured p90 lead time, with monthly drift review and Finance choosing own stock versus consignment.

R — Result. This design yields one auditable date chain and one capacity signal: receipts on/before confirmed date ≥95%, valid ASNs ≥95%, unconfirmed-after-48-hours → 0, receipts without ASN → 0. An illustrative avoided-idle-week value is ~$50K; I would ask Finance for the actual crew/deferred-activation cost before sizing impact.

TRADE-OFF I MADE. I would delay activation for weak-data suppliers and let them use a portal template until they graduate. That costs speed in month one but prevents a bad ASN from contaminating serial, receipt, capacity, and Finance downstream. For fixed sites I would use project stock; for pooled equipment I would preserve aggregation rather than fragment inventory by project.

LIVED FAILURE / LEARNING. In my real lane onboarding, completeness was instrumented from day one but tender acceptance only from cohort three, so a carrier-data issue surfaced two cohorts late. I now require both "message is valid" and "business transaction succeeded" before a vendor graduates.

LP tags. Ownership—one owner and exception clock per PO line. · **Dive Deep**—request and confirmation are separate causal metrics. · **Insist on the Highest Standards**—the ASN gate blocks activation. · **Deliver Results**—expediting ranks the milestone at risk, not the loudest requester.

SAFE-SAY. "Twelve-to-forty weeks and any dollars here are illustrative design inputs; the one-thousand-lane/597-first EDI wave and its data-quality gate are my lived record."
"I have not run VMI; I would own its process and controls while Procurement, Finance, and SAP owners execute their parts."

20-SECOND VERSION

I would turn long-lead buying into one dated chain from build signal to confirmed PO, serialized ASN, receipt, and invoice. Vendors graduate only after valid messages and successful transactions; requested-versus-confirmed gap is separate from vendor reliability; qualified VMI uses measured tails for min/max. The capacity signal is confirmed supply, not hope.

FOUR PROBES

1 · What if an 855 says "accepted" without a date?

It is unconfirmed, visible after 48 hours, and excluded from trusted capacity. "Acknowledged" is transport success, not a supply commitment.

2 · Own stock or consignment?

Model both. Consignment keeps vendor ownership until consumption and helps working capital; own stock simplifies accounting. Finance chooses explicitly.

3 · How do you prioritize scarce supply?

Milestone-at-risk × customer/build impact, constrained by qualification and committed projects—not who escalates first.

4 · What is the kill switch?

Disable the vendor map/cohort, preserve the last valid confirmation, route new transactions through the portal, and keep receipts blocked until serial/quantity reconcile.

DESIGN CASE — E1 + E2 · OUTBOUND CONNECTIVITY + CROSSDOCK ENABLEMENT

J4B · Contractor truck stock → serial-number chain from warehouse to premise

Lived analog: owner-partitioned inventory and audited transitions in Sephora; real RF-device readiness miss. **Primary process metric:** unaccounted units per contractor and complete serial chain.

60-SECOND PITCH

S — In the field-inventory design, the 3PL saw a shipment, the contractor saw a van, CRM saw an assigned technician, and Finance saw stock—but no one could prove where a serialized unit sat. **T —** I would make every truck a governed node and every install an auditable state transition. **A —** I would model 3PL, crossdock, van, installed, and returns as explicit stocks; scan serials on ASN, issue, transfer, install, swap, and return; let CRM read only the assigned tech's available stock; replay offline events by serial with idempotency; gate replenishment on a posted weekly count; and reconcile SAP, 3PL, field app, and CRM daily. **R —** Proposed bars are ≥99% same-day install posts, 100% weekly counts, zero unexplained serials, and zero "van said yes, book said no" reschedules. Volumes and dollars remain illustrative.

S — Situation. I would treat "unaccounted" as an undocumented flow before calling it loss. A truck is inventory outside a warehouse; without a location, serial state, and document, the system guarantees phantom availability.

T — Task. I would own the state model, node template, event contract, exception queue, crossdock decision, count policy, and activation checklist. Field Ops owns counts; Inventory Control owns the book; SAP/3PL/CRM teams build.

A — Action. I would create a location per contractor/truck; 3PL→van is a scanned transfer, van→premise an install by serial, swap a paired scan-out/scan-in, and return a move to untested stock. Crossdock applies when a node kit is preassigned and ASN/serial/order all match; otherwise it enters 3PL stock. Every event uses document+line+serial, offline events replay in serial order, unknown/wrong-location installs hold, counts post adjustments only with reason, and a missed weekly count blocks new issues with a direct-fulfillment escape route.

R — Result. The design makes the complete serial chain the quality metric: ASN→receipt→van→install/CRM asset→return. Targets: serial-state accuracy ≥99.5%, same-day install posting ≥99%, posted weekly counts 100%, unaccounted near zero. Illustrative sizing in the design is 500 vans × two unexplained units/month × $150 ≈ $1.8M/year of stock without a reliable owner—not claimed savings.

TRADE-OFF I MADE. Per-serial scanning and count-as-gate slow receiving and weekly replenishment; I accept that cost because silent phantom stock creates failed installs and unauditable chargebacks. Crossdock removes touches only when identity is already proven; otherwise the 3PL validation step stays.

LIVED FAILURE / LEARNING. On a real DC migration, device scripts for hundreds of RF devices were not proven before training; training moved 1.5 weeks, although go-live held. I learned physical/device readiness must be an entrance criterion with a named owner, not a task buried under software readiness.

LP tags. Invent and Simplify—one state machine across four systems. · **Highest Standards**—wrong-location serials hold rather than auto-fix. · **Bias for Action**—offline capture plus ordered replay keeps field work moving. · **Ownership**—count and chargeback responsibilities are explicit.

SAFE-SAY. "The ≥99.5% bar and van-dollar example are illustrative design targets; the RF-device delay and owner-partitioned inventory controls are lived records."
"I designed the state and integration contract; I did not configure the SAP serial profile."

20-SECOND VERSION

I would make every van a governed inventory node and every serial move a document: ASN, 3PL receipt, van issue, install/CRM asset, swap, and return. Crossdock is allowed only on a three-way identity match; offline events replay idempotently; missed counts stop replenishment. The result is a complete serial chain, not a better spreadsheet.

FOUR PROBES

1 · Why a location, not a spreadsheet?

The client owns the unit and Finance must locate it. A spreadsheet creates a second book; a governed location makes transfer, count, and adjustment auditable.

2 · How does crossdock stay safe?

PO/ASN serial, node kit, work order, and assigned technician must match before bypassing storage; any mismatch routes to controlled 3PL receipt.

3 · Field app is offline—then what?

Mint and persist an idempotency key on device, timestamp, replay in order per serial, and expose "captured not posted" as timing—not loss.

4 · What prevents false chargebacks?

Only a posted physical-inventory document with reason can feed chargeback. Missing events trigger reconciliation and a recount, not an accusation.

NEW-NODE ACTIVATION CARD

LAYER	REQUIRED BEFORE VOLUME	ESCAPE ROUTE
Identity	Node/location, contractor, truck, kit, and serial policy configured.	No identity match: receive to controlled 3PL stock.
Connectivity	ASN, issue, install, swap, return, offline replay all pass.	Freeze node issues; fulfill direct from 3PL.
Physical	Devices proven; first count posted; capacity/route confirmed.	Manual scan/count path, not silent spreadsheet.
Audit	Three-way serial reconciliation clean.	Hold wrong-state serial; reason-code and correct.

DESIGN CASE — E5 · POST-DEPLOYMENT VALIDATION + AUDITABILITY

J4C · 3PL reconciliation — “two books, one truth”

Lived analog: three production-scale mock reconciliations moved variance 7%→1.8%→0.3% against a pre-published 0.5% tolerance (record). **Placement hook:** lower-of-two promise basis while configuration drift is unresolved.

60-SECOND PITCH

S — In the design, SAP holds the financial/storage-location book while a 3PL WMS holds the physical bin/serial truth; timing, status, and unit-of-measure differences make the delta look like shrink. **T —** I would protect customers and Finance without letting either system “win” by assertion. **A —** I would align SKU×location×state×serial, compare daily snapshots against an in-flight ledger, classify every delta as timing, mapping, status, or physical, promise from the lower of SAP/WMS until resolved, and allow only physical differences to become referenced adjustments. Finance publishes tolerance before the first comparison; serials have zero tolerance. **R —** The proposed result is consecutive days inside tolerance, zero unexplained serials, 100% reason-coded adjustments, and no silent month-end true-up. The lived proof is the mock arc that closed at 0.3% against 0.5%.

S — Situation. I would start where two valid books disagree at different grains: WMS bin/status/serial versus SAP location/stock type. Ad hoc adjustment hides whether the defect is a late message, bad map, wrong state, or physical loss.

T — Task. I would own the grain contract, status/UoM maps, in-flight rules, tolerance, reason taxonomy, owner/SLA, promise safety rule, and close control. Finance signs tolerance; the 3PL owns physical evidence; Inventory Control owns posting.

A — Action. I would ingest daily 846 inventory plus 944 receipts, 945 ship confirms, and 947 adjustments; roll bins to locations and compare serials as sets; subtract expected in-flight documents before calling variance; quarantine unknown statuses rather than map them to unrestricted; route timing to integration, mapping to master data, status to operations, and physical to count/research; expose aging and owner; post 701/702 only from a unique 3PL adjustment id; keep promising on min(SAP,WMS) until closure.

R — Result. Designed bars: rows outside tolerance → 0 on consecutive days, unexplained serials = 0, adjustments with reason = 100%, unposted confirms over 48 hours = 0. Illustrative sizing: on $20M held at 3PL, moving unexplained delta from 2% to 0.5% resolves about $300K of uncertainty—not claimed savings.

TRADE-OFF I MADE. Lower-of-two under-promises while a delta stands, and zero serial tolerance creates exceptions. I accept temporary capacity loss over a customer promise against phantom stock or an unexplained asset at close.

LIVED FAILURE / LEARNING. In my real migration, mock one was 7% off because of UoM, location, and status vocabulary; mock two reached 1.8% and exposed in-transit timing; a separate in-transit batch and 72-hour freeze brought mock three to 0.3%. I learned never to celebrate a shrinking total without a named cause class.

LP tags. Earn Trust—Finance sets the tolerance before results. · **Dive Deep**—the delta is decomposed by cause. · **Customer Obsession**—lower-of-two blocks phantom promises. · **Highest Standards**—serial uncertainty has zero tolerance.

SAFE-SAY. “7%→1.8%→0.3% against 0.5% is the lived migration record; the $20M/$300K line is illustrative design sizing.” “The source is master E5 paired with the recorded mock-cutover reconciliation; I claim the control design, not a Google Fiber outcome.”

20-SECOND VERSION

I reconcile SAP and the 3PL at aligned grain, subtract in-flight documents, classify the remainder into timing/mapping/status/physical, and promise from the lower book until resolved. Only physical differences become controlled adjustments. The lived analog moved three mocks from 7% to 0.3% against a 0.5% gate.

FOUR PROBES

1 · Which book wins?

Neither automatically. SAP is Finance’s book, WMS is physical truth; the reason-coded investigation decides. Promising uses the lower while they disagree.

2 · How do you prevent replayed adjustments?

Snapshot key = date+warehouse; adjustment id is unique and referenced by the SAP document. Replay returns the existing post.

3 · What if an unknown status arrives?

Hold it as unknown and unpromisable. Never default an unmapped state to unrestricted.

4 · What makes this governance, not reporting?

The report changes behavior: lower-of-two constrains promise, aging escalates owners, tolerance gates close, and only a governed cause can post an adjustment.

DESIGN CASE — E10 · ESCAPE ROUTING / REVERSE FLOW

J4D · Returns/RMA — premise test → refurb / RTV / scrap

Lived analog: contract-manufacturer disposition forum and a real partial-extract failure that looked like mass deletion. **Architecture hook:** exceptions do not bypass the main process; they enter a governed state machine.

60-SECOND PITCH

S — In the reverse-flow design, disconnected and swapped equipment could reach a returns bin, then disappear into "returned" without test, grade, warranty recovery, or a reliable serial state. **T —** I would create escape routing that preserves customer speed without turning exceptions into unaudited shortcuts. **A —** I would open an RMA by serial/reason, receive into returns-untested, consolidate to a standard test cell, and let the test event route the same serial to refurb A/B, repair, blocked failure, vendor return, or certified scrap. No test means no grade and no promise; warranty and vendor-RMA rules block the wrong disposition; aging and expected-versus-received credits sit in one weekly forum. **R —** Proposed bars are ≥95% disposition within 14 days, ≥80% warranty recovery on eligible failures, zero untested units made available, and 100% scrap certificates. All economics are illustrative.

S — Situation. I would treat "returned" as a flow, not a final state. A unit can be physically back but still unsafe to promise, financially eligible for vendor recovery, or absent from the book because the field event never posted.

T — Task. I would own the legal states/transitions, reason/grade vocabulary, SLA, warranty logic, escape paths, credit reconciliation, and disposition forum. Service owns the case; the test partner owns evidence; Finance owns write-off/credit.

A — Action. I would require serial+reason at initiation; post pickup/tech return into untested stock; preserve identity through consolidation; accept test results only for serials in the expected state; use grade as an attribute, never renumber; compute warranty from the serial's contract start event; require vendor RMA before shipping RTV; prevent in-warranty scrap unless Finance approves; reconcile return initiated→received→tested→dispositioned and expected→received credit; escalate aging by state and reason.

R — Result. The designed result is a measurable escape route: ≥95% within 14 days, warranty recovery ≥80% of eligible failures, untested available = 0, RTV without vendor RMA = 0, scrap certificate = 100%. A 48K-return/$3.1M-net refurb case in E10 is **illustrative**, not a delivered claim.

TRADE-OFF I MADE. I chose centralized, standardized testing over faster grading on every truck. It adds a consolidation hop and days, but avoids inconsistent grades and a second truck roll caused by promising an untested unit. During surges, availability waits; quality does not.

LIVED FAILURE / LEARNING. In my real contract-manufacturer data program, an unflagged partial extract looked like thousands of deleted components. I added a completeness flag as a hard stop only by wave three—later than I should have. The standing disposition call then caught a wave-two yield gap as one mapping row. That is why incomplete return/test feeds cannot advance state here.

LP tags. Customer Obsession—untested stock never reaches a premise. · **Ownership**—each disposition and credit has an owner. · **Invent and Simplify**—one serial state machine replaces side queues. · **Frugality**—refurb and warranty recovery avoid needless new units.

SAFE-SAY. "The 14-day/80% bars and all return economics are illustrative design targets; the completeness-flag miss and weekly disposition forum are lived."
"I have not run an RMA program end to end; I have run the state, data-quality, and disposition-control analog."

20-SECOND VERSION

I would route every returned serial through one legal chain: RMA, untested receipt, standard test, then refurb/repair/RTV/scrap. No complete feed, no state advance; no test, no promise; no vendor RMA, no return shipment. Aging and credits close in one disposition forum.

FOUR PROBES

1 · Why keep the same serial after refurb?

Identity and warranty history must remain continuous. Grade changes; the serial does not.

2 · What is escape routing here?

A failed normal transition enters a named state with an owner and legal exits—it never jumps directly to unrestricted, write-off, or a spreadsheet queue.

3 · How do you stop warranty leakage?

Eligibility derives from the serial's contract start event; in-warranty scrap is rule-blocked; RTV requires vendor authorization; expected credits reconcile to receipts.

4 · What stops an incomplete file?

A completeness manifest with counts and sequence; absent/partial manifests hold the whole batch. Partial truth cannot move serial state.

DESIGN CASE — E12 · NODE CAPACITY SIGNALING + CONSIGNMENT LEDGER

J4E · A tech van as a vending machine — custody, consumption, replenishment, settlement

Lived analog: idempotent inventory decrement/reconciliation and a real status-mapping failure in which legal events did not advance state. **Key idea:** the van is the machine, install is dispense, refill is replenishment, kit is planogram.

60-SECOND PITCH

S — In the Google Fiber design, hundreds of field nodes can hold vendor-owned equipment, work offline, consume at install, and need replenishment before the next route. A receipt proves custody, not liability. **T —** I would create one event ledger that supports inventory, capacity, and Finance without double-consumption. **A —** I would receive consignment by serial, mint an install event at the edge before state changes, replay idempotently after reconnect, post consumption only at verified install, set van par from demand over review+lead time plus variability, rank refills by stockout risk×customer impact, reconcile every count to the ledger, and settle consumed units monthly. **R —** Proposed bars are ≥98% fill, zero duplicate consumption, 100% count-backed adjustments, and ledger backlog drained within ten minutes of reconnect. The working-capital example is illustrative, not claimed.

S — Situation. I would start with a configuration problem disguised as inventory: node capacity, ownership, offline events, and refill policy differ by contractor and market, so uncontrolled local defaults create both stockouts and settlement disputes.

T — Task. I would own the node template, event/state contract, par policy, refill priority, offline/duplicate controls, reconciliation, and settlement rule. Field Ops owns counts; Planning owns demand; Finance and Procurement own consignment terms.

A — Action. I would register each van/node with capacity, kit/planogram, owner, route, and connectivity policy; receive vendor-owned stock into a consignment state; persist event id = node×slot/SKU×sequence before install; post consumption once and create the CRM asset; use observed demand and measured replenishment lead time for par capped by physical capacity; signal overflow by shortening route interval rather than inventing capacity; compare pre-refill count to ledger and reason-code shrink; settle consumed units, delivery notes, and counts as a three-way match.

R — Result. Designed outcome: fill ≥98%, duplicate consumption = 0, count-backed adjustments = 100%, online ledger latency p95 under 60 seconds, reconnect drain under 10 minutes. Illustrative working-capital sizing—500 vans×30 units×$150 ≈ $2.25M held vendor-owned until install—is a Finance decision aid, not a result.

TRADE-OFF I MADE. Consignment defers cash but adds ownership states, settlement, and supplier reconciliation. I would use it for expensive, stable, high-volume kit components; I would keep volatile or low-value items as owned stock. If physical capacity caps par, I change route cadence—not hide the shortage in a higher target.

LIVED FAILURE / LEARNING. In a real visibility program, events arrived legally but the scraper and application used different status vocabularies, leaving roughly half the containers stuck in “verification in progress.” I traced the hops, added a mapping layer and contract tests. Here, unknown install/consumption states hold and alert; they never default to consumed or available.

LP tags. Think Big—one node pattern scales across a field fleet. · **Frugality**—consignment is an explicit working-capital lever. · **Dive Deep**—capacity and route cadence are separated. · **Earn Trust**—the supplier settlement is reproducible from events and counts.

SAFE-SAY. “The 98% bar, latency bars, and $2.25M example are illustrative; I have not run a consignment or vending program.”
“My lived evidence is the idempotent decrement/reconciliation pattern and the status-vocabulary failure; I claim the architecture and controls.”

20-SECOND VERSION

I treat a van like a vending node: receipt creates custody, install creates consumption and liability, count reconciles the ledger, and refill follows measured demand and route lead time. Events are edge-minted and idempotent; unknown states hold; monthly settlement reproduces from consumption, delivery notes, and counts.

FOUR PROBES

1 · Why consignment?

For expensive predictable items, it shifts working capital until install. It is not free: settlement and reconciliation complexity must be worth the capital benefit.

2 · How do you handle offline ordering?

Persist a monotonic node sequence before the physical event, replay in order, dedupe server-side, and expose backlog age; never order by wall-clock.

3 · What if par exceeds capacity?

Capacity is a hard constraint. Shorten route interval, re-slot the kit, or add a node; do not publish an impossible par.

4 · How does this signal a new node type?

The node template carries kit, capacity, demand class, route, ownership, and connectivity policy; rollout is blocked until its event, count, and settlement paths validate.

CAPACITY-SIGNAL CARD

SIGNAL	DECISION	GOVERNANCE CHECK
Demand vs par	Refill normal route.	Observed installs, not requested kit, drive usage.
Par > physical capacity	Shorten interval, re-slot, or add node.	Impossible par cannot publish.
Ledger ≠ count	Hold settlement delta; reason-code shrink.	Count changes book only through a document.
Backlog / stale heartbeat	Conservative availability; drain ordered events.	Unknown state never defaults to available or consumed.